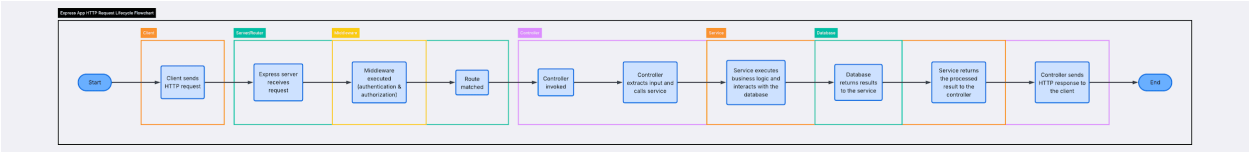
System Architecture and Request Lifecycle

The backend follows a layered architecture to ensure separation of concerns and maintainability. The system is organized into routes, controllers, services, middleware, and database interaction logic. Each layer has a distinct role. Routes define the HTTP endpoints and map incoming requests to specific controller functions. Controllers handle request level concerns such as extracting parameters, performing basic validation, and preparing responses. They in most cases do not implement complex business logic. Instead, they delegate core operations to the service layer. The service layer contains the main application logic, including rule enforcement, transaction handling, and database operations. Database interaction is performed using raw SQL queries through a connection pooling mechanism.

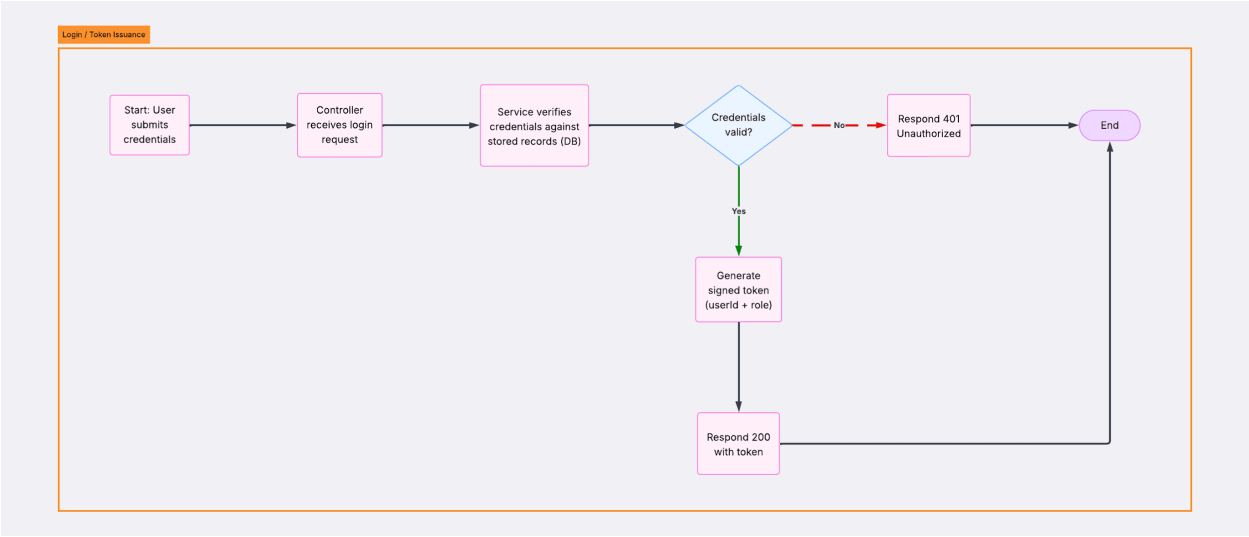


When a request is processed, it follows a structured lifecycle

1) The HTTP request reaches the Express server.
2) Middleware gets executed (authentication and authorization checks).
3) The request is matched to a defined route.
4) The corresponding controller function is invoked.
5) The controller extracts input data and calls the appropriate service function.
6) The service executes business logic and interacts with the database.
7) The database returns results to the service.
8) The service returns the processed result to the controller.
9) The controller sends the final HTTP response to the client.

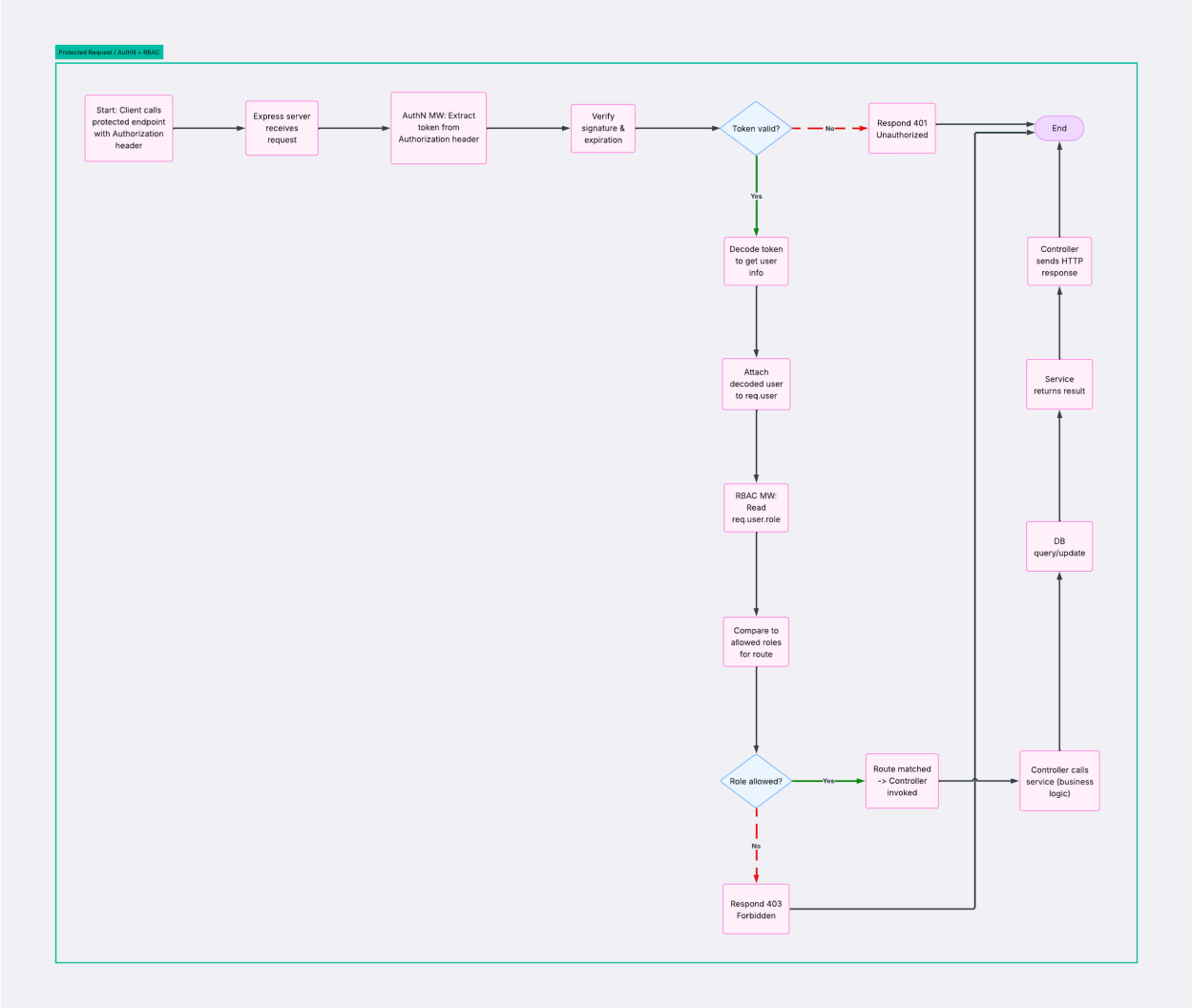
Authentication Flow and RBAC Enforcement

Authentication is implemented using a token-based mechanism. When a user submits valid credentials, the server verifies them against stored records. If the credentials are correct, a signed token is generated and returned to the client. The client must include this token in the authorization header of all next protected requests.



For protected endpoints, the request passes through authentication middleware before reaching the controller. The middleware performs the following steps:

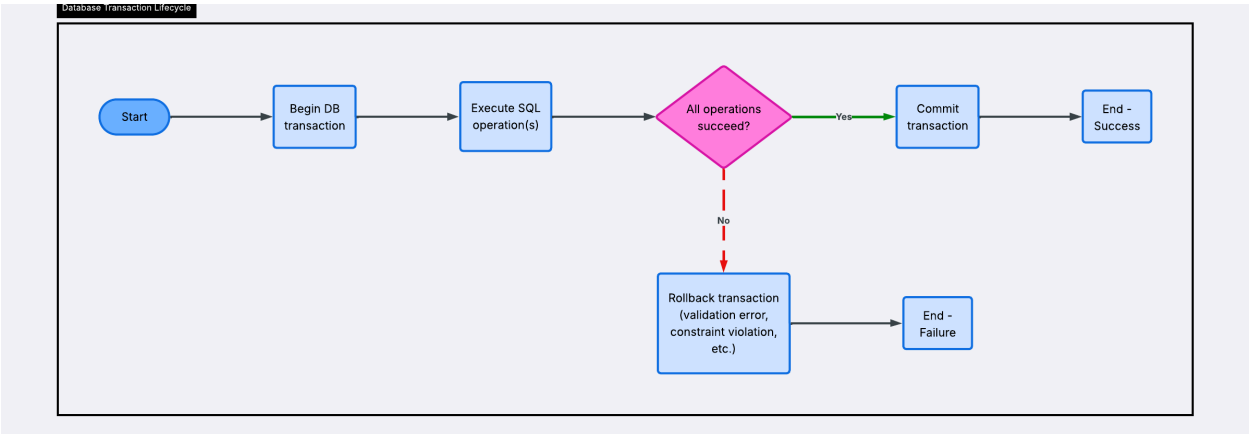
1) Extract the token from the request header.
2) Verify the token signature and expiration.
3) Decode the token to retrieve user information.
4) Attach the decoded user data to the request object.
5) Allow the request to proceed if valid. Otherwise, reject it.



After authentication, Role Based Access Control (RBAC) is enforced. Each protected route specifies which roles are allowed to access it. The authorization middleware checks the user’s role from the decoded token and compares it against the required permissions. If the role does not match the allowed set, the request is denied.

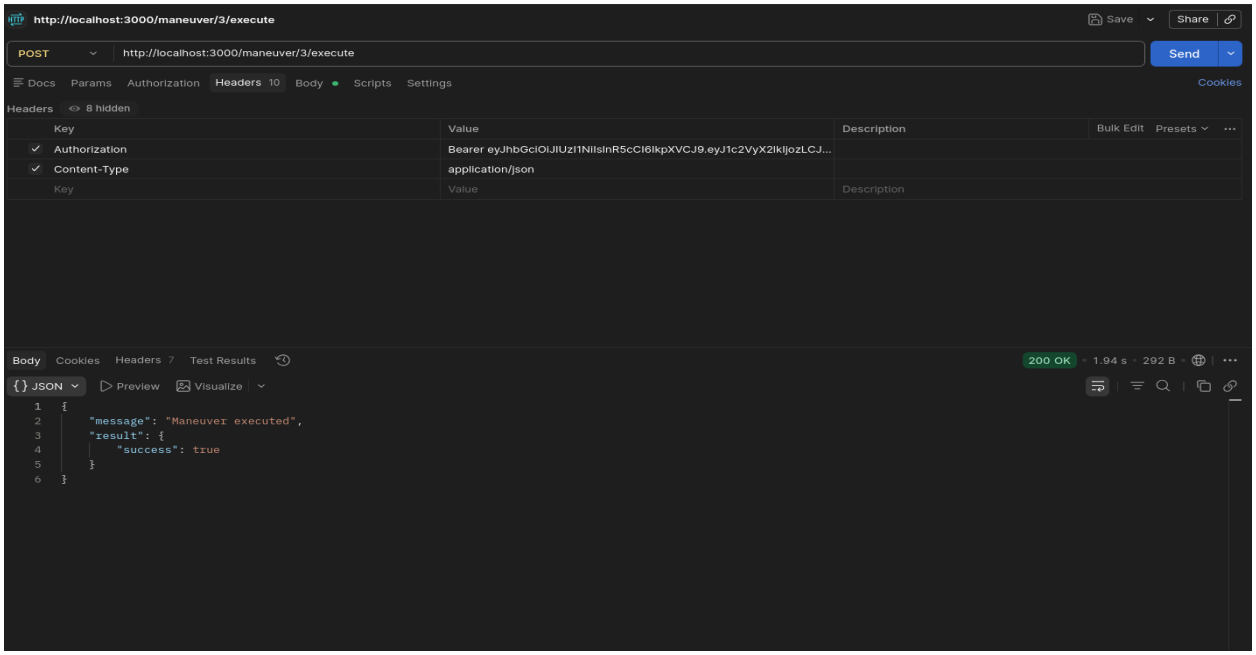
Transaction Flows and Rollback Conditions

When a transactional operation is required, the following steps occur:

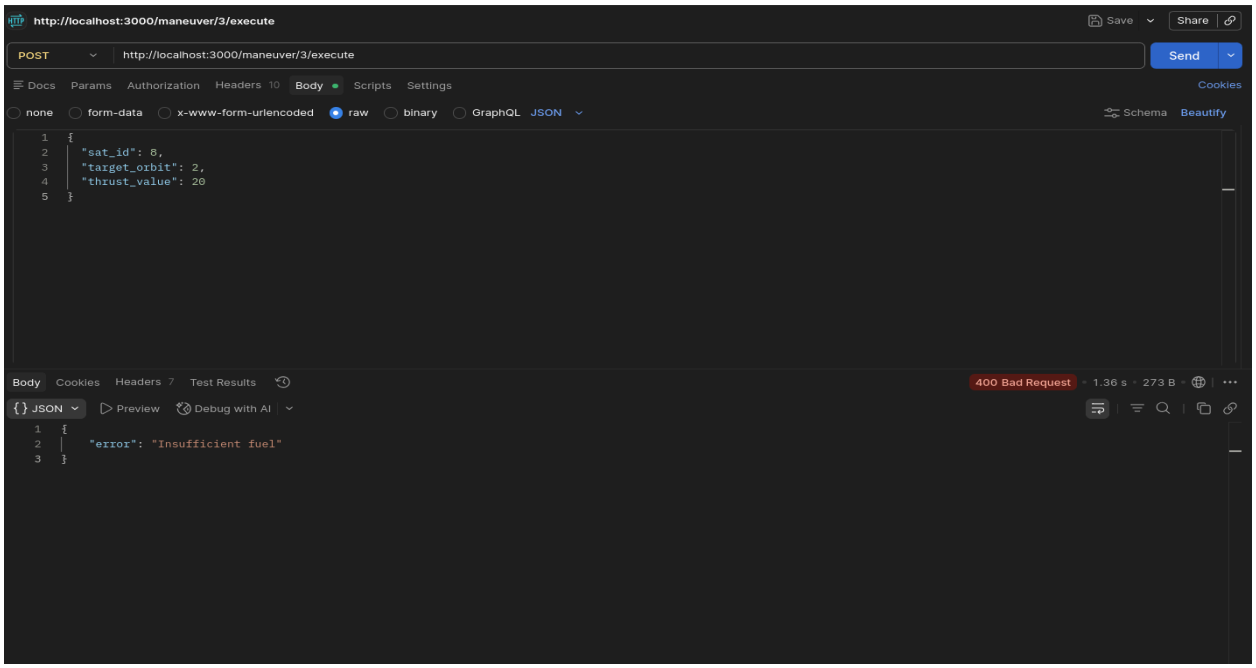


1) Begin a database transaction
2) Execute the SQL operation
3) If all operations succeed, commit the transaction
4) If any operation fails due to an error such as validation issue or constraint violation, rollback the transaction

Transaction Scenario: Promoting a satellite to higher orbit



In case of insufficient fuel, there is constraint violation so rollback occurs.



(Detailed flow of both of these cases is in media/ directory)

Connection Pooling and Raw SQL Strategy

The system uses a connection pool to manage database connections efficiently. Instead of opening and closing a new connection for each request, the application maintains a pool of reusable connections. When a database operation is required, a connection is acquired from the pool then used for the query, and then returned to the pool. This strategy reduces overhead.

Instead of ORM for critical transactions, Raw SQL queries are used to improve performance by having low level control.

Key Design Decisions and Tradeoffs

1) One major architectural decision was the adoption of a layered architecture instead of a tightly coupled logic design. While this approach increased the number of files and introduced additional structural complexity, it significantly improved separation of concerns. By isolating routes, controllers, services,and database logic into distinct layers, the system achieved clearer responsibility boundaries and improved scalability.
2) Another important design consideration involved the level of business logic complexity implemented in the system. A more advanced and detailed business logic model could have increased realism and closer alignment with real world operational behavior however implementing extremely complex business logic would have increased development time and system complexity potentially reducing overall project clarity and completeness within the given scope. That is why a balanced level of business logic is implemented to ensure functional accuracy and manageability of project.
3) Additionally the simulation model simplifies orbital representation by projecting three dimensional space onto a two dimensional plane. In reality orbital dynamics occur in three dimensional space, modeling which would have introduced significantly higher mathematical complexity. By limiting the system to a two dimensional orbital plane the number of orbits remain finite and hence manageable. This decision also improves clarity of visualization while intentionally trading off physical realism.